HotRelay

SPÉCIFICATION TECHNIQUE

HRP

HotRelay Protocol

Le langage commun des téléphones HotRelay : rejoindre une session, connaître les participants et transférer des fichiers bloc par bloc sur UDP, sans Internet.

VERSION	DATE	TRANSPORT	STATUT
1.0	12 novembre 2026	UDP · port 50000	Figée · à valider par les tests

Projet HotRelay · SUP'PTIC · Équipe HotRelay

HRP en un coup d'œil

UDP Transport · port 50000	**8 octets** En-tête commun, MAGIC « HRP »	**14** Types de messages
1 024 o Données max. par bloc	**8** Participants max. (hôte + 7)	**Stop-and-Wait** Un bloc, un accusé NICE

Ce que fait HRP

UDP SEUL NE GARANTIT PAS…

- × que le paquet arrive
- × qu'il arrive une seule fois
- × qu'il arrive dans l'ordre
- × qu'on sache à qui il s'adresse

…DONC HRP AJOUTE

- des accusés de réception (NICE) et des retransmissions
- la détection des doublons
- des numéros de bloc (SEQ) pour l'ordre
- des identifiants de participants et de transferts
- une empreinte SHA-256 pour vérifier le fichier final

Sommaire

1. La pile réseau
2. Architecture : l'hôte relais
3. Identifiants
4. Conventions binaires
5. L'en-tête commun
6. Registre des messages
7. Détail des messages
8. Session et participants
9. Transfert de fichiers
10. Retransmissions et délais
11. Erreurs et interruptions
12. Robustesse et sécurité
13. Décisions v1 et versionnage
14. Annexes : structures, Java, tests

1. La pile réseau

HRP est un **protocole applicatif** : il ne remplace ni UDP ni IP, il s'appuie dessus. Chaque couche a un seul travail. HRP s'occupe de tout ce qu'UDP ne fait pas.

Application HotRelay
Écrans, choix du fichier, affichage de la progression

HRP · HotRelay Protocol
Session, participants, découpage en blocs, NICE, retransmissions, SHA-256

UDP
Datagrammes, ports · 50000 côté hôte

IP
Adresses du réseau créé par le hotspot

Wi-Fi 802.11
Point d'accès activé manuellement sur l'hôte

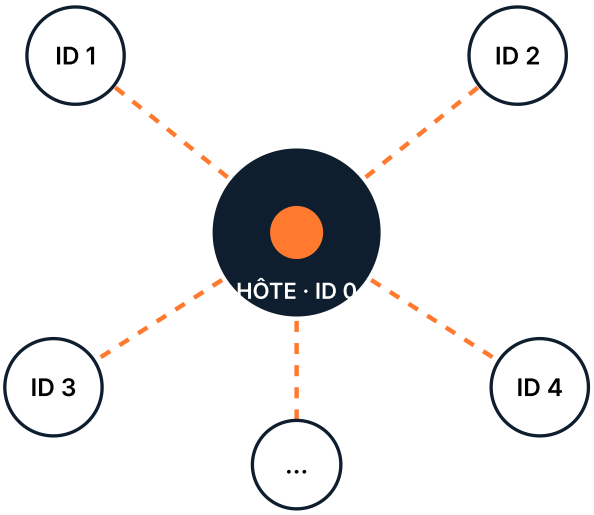
ENCAPSULATION D'UN MESSAGE HRP



Le plus gros message (TAKE) fait 8 + 12 + 1 024 = **1 044 octets** de données UDP, sous la limite de ~1 472 octets d'un réseau Wi-Fi classique : **pas de fragmentation IP**.

À retenir — UDP, c'est la poste : il transporte des cartes postales sans rien garantir. HRP, c'est ce qu'on écrit sur les cartes et les règles pour s'assurer que le fichier arrive entier.

2. Architecture : l'hôte relais



Un téléphone active son hotspot et lance HotRelay en mode **hôte**. Les autres rejoignent ce Wi-Fi puis la session : ce sont les **clients**.

Les clients ne se parlent **jamais directement**. Tout passe par l'hôte, qui lit l'en-tête, consulte sa table et relaie le datagramme.

TABLE DE SESSION (HÔTE)

ID	Nom	Endpoint	Dernier signe
0	Hôte	—	—
1	Paul	192.168.x.12:41822	il y a 2 s
2	Aïcha	192.168.x.87:38004	il y a 4 s

Rôle	Responsabilités
Hôte · ID 0	Crée la session · attribue les IDs · tient la table (nom, IP, port, dernier signe de vie) · valide et relaie les messages · diffuse SYNCTABLE · peut aussi envoyer et recevoir des fichiers
Client · ID 1 à 7	Rejoint la session · envoie AVAILABLE · choisit un destinataire · envoie ou reçoit des blocs · reconstruit le fichier

L'adresse de l'hôte — Depuis Android 10, l'IP du hotspot n'est pas toujours 192.168.43.1. L'écran hôte affiche l'adresse réellement lue sur l'interface ; le client la saisit à la main (pas de découverte automatique en v1).

L'hôte **ne stocke pas** les fichiers : il garde seulement le routage de chaque transfert (SRC, DST, TRANSFER_ID, état). C'est l'émetteur qui conserve chaque bloc jusqu'à son NICE.

3. Identifiants

Quatre identifiants, chacun avec une portée précise. Les confondre est la source d'erreur la plus fréquente.

Identifiant	Taille	Qui le crée	À quoi il sert
`REQUEST_ID`	8 o	Le client, au hasard, une fois par tentative	Reconnaître une même demande NEW renvoyée plusieurs fois, **avant** que le client ait un ID
`ID` participant	1 o	L'hôte, dans JOIN	Désigner un participant : 0 = hôte, 1 à 7 = clients. Jamais réutilisé dans une session
`TRANSFER_ID`	4 o	L'émetteur du fichier	Désigner un transfert complet ; permet à l'hôte de router les TAKE
`SEQ`	4 o	L'émetteur	Numéro d'un bloc **dans** un transfert (0, 1, 2…)

Construction du TRANSFER_ID

Le message TAKE ne répète ni l'émetteur ni le destinataire : l'hôte route uniquement grâce au TRANSFER_ID. Deux transferts simultanés ne doivent donc **jamais** porter le même numéro. La règle :

```
TRANSFER_ID = (ID de l'émetteur × 2^24) + compteur, compteur = 1, 2, 3…
```

ID émetteur 1 octet	Compteur propre à l'émetteur 3 octets · 1 à 16 777 215

`01` `00 00 01` — Paul (ID 1) 1er envoi

`02` `00 00 01` — Aïcha (ID 2) 1er envoi

Paul et Aïcha comptent chacun de leur côté sans jamais entrer en collision : le premier octet diffère. Le compteur démarrant à 1, un TRANSFER_ID vaut **jamais 0** (même pour l'hôte, ID 0 → `00 00 00 01`), ce qui libère la valeur 0 pour « aucun transfert » dans ERROR. L'hôte vérifie que le premier octet correspond bien à l'émetteur.

4. Conventions binaires

Type	Taille	Représentation
U8	1 o	Entier non signé 0 à 255
U16	2 o	Non signé, big-endian
U32	4 o	Non signé, big-endian
U64	8 o	Non signé, big-endian
BYTES	variable	Suite d'octets brute
UTF-8	variable	Texte ; longueur comptée en **octets**

Big-endian (ordre réseau) : l'octet de poids fort part en premier. 1 036 s'écrit `04 0C`.

```
ByteBuffer buf = ByteBuffer.wrap(data)
    .order(ByteOrder.BIG_ENDIAN);
```

Une longueur de texte compte des octets, pas des caractères : `« Aïcha »` fait 5 caractères mais **6 octets** en UTF-8.

Les entiers Java étant signés, lire un U8 avec `b & 0xFF`, un U16 avec `getShort() & 0xFFFF`, un U32 avec `getInt() & 0xFFFFFFFFL`.

5. L'en-tête commun

Tous les messages HRP commencent par le même en-tête de **8 octets**, suivi du payload propre au type.



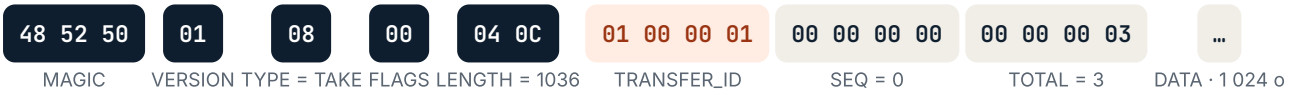
Octets 0 à 7 : en-tête · à partir de l'octet 8 : payload

Octets	Champ	Valeur en v1	Rôle
0–2	MAGIC	`48 52 50` (« HRP » en ASCII)	Reconnaître un datagramme HRP et jeter tout le reste
3	VERSION	`01`	Version du protocole ; une version inconnue est ignorée
4	TYPE	`01` à `0E`	Dit comment lire le payload
5	FLAGS	`00` obligatoire	Réservé. En v1, un FLAGS ≠ 0 rend le paquet invalide : il est ignoré
6–7	LENGTH	0 à 1 036	Longueur du **payload seul**, U16 big-endian

```
Taille du datagramme = 8 + LENGTH
```

Exemple : décoder un TAKE

Paul (ID 1) envoie le bloc 0 de son premier transfert, un fichier de 3 blocs. Bloc plein : 12 octets de métadonnées + 1 024 de données → LENGTH = 1 036 = `04 0C`.



Validation, dans cet ordre

1. Le datagramme fait au moins **8 octets**.
2. MAGIC = `48 52 50`, VERSION = `01`, FLAGS = `00`.
3. TYPE est connu (sinon : ignorer sans essayer de lire le payload).
4. Taille réelle = **8 + LENGTH** exactement (ni tronqué, ni octets en trop).
5. LENGTH respecte la taille minimale du TYPE ; chaque champ de longueur est vérifié avant lecture.

Règle d'or — un paquet réseau est une entrée non fiable, même sur un réseau local. Un message invalide est ignoré : il ne doit **jamais** faire planter l'application.

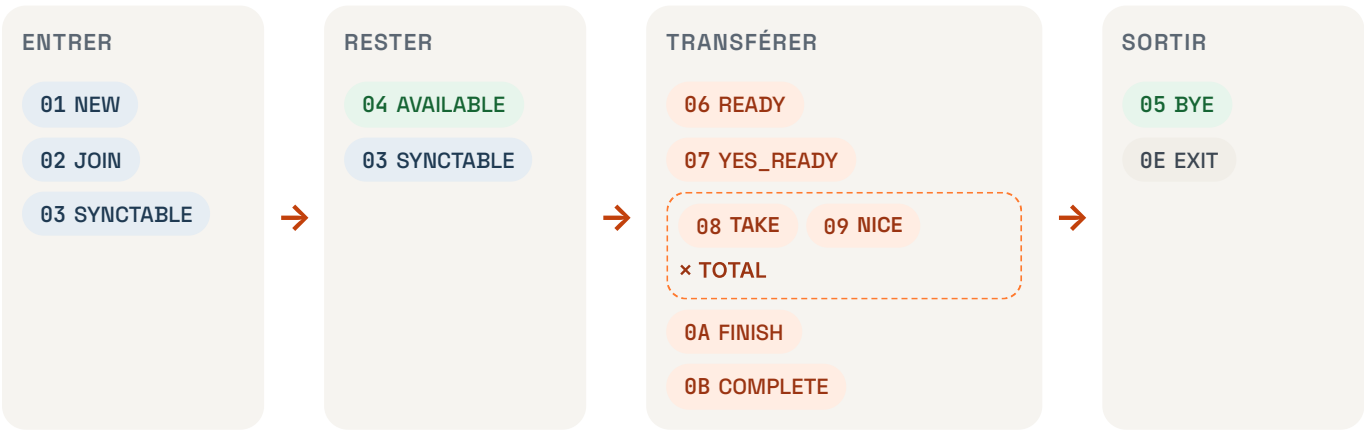
6. Registre des messages

Quatorze types répartis en quatre familles. Un TYPE inconnu est ignoré sans crash.

Session · Présence · Transfert · Contrôle

Code	Message	Direction	Rôle
0x01	**NEW**	Client → hôte	Demande de rejoindre la session
0x02	**JOIN**	Hôte → client	Admission et attribution de l'ID
0x03	**SYNCTABLE**	Hôte → tous	Liste des participants · signe de vie de l'hôte
0x04	**AVAILABLE**	Client → hôte	Signe de vie du client (5 s)
0x05	**BYE**	Client → hôte	Départ volontaire
0x06	**READY**	Émetteur → hôte → dest.	Annonce d'un fichier
0x07	**YES_READY**	Dest. → hôte → émetteur	Acceptation
0x08	**TAKE**	Émetteur → hôte → dest.	Un bloc de données
0x09	**NICE**	Dest. → hôte → émetteur	Accusé de réception d'un bloc
0x0A	**FINISH**	Émetteur → hôte → dest.	Fin + empreinte SHA-256
0x0B	**COMPLETE**	Dest. → hôte → émetteur	Fichier reçu et vérifié
0x0C	**ERROR**	Selon le cas	Signalement d'une erreur
0x0D	**CANCEL**	Émetteur → hôte → dest.	Annulation d'un transfert
0x0E	**EXIT**	Hôte → tous	Fermeture de la session

Le cycle de vie, d'un coup d'œil



7. Détail des messages

Chaque schéma montre le payload placé après l'en-tête de 8 octets. Les champs orange sont ceux qui servent au routage ou à la corrélation.

0x01 NEW

Client → hôte

Le client demande à rejoindre la session.

En-tête	REQUEST_ID	NAMELENGTH	NAME
8 o	8 o · U64	1 o	≤ 32 o UTF-8

- Le même REQUEST_ID est réutilisé à chaque renvoi de cette demande.
- Session pleine (8 participants) → ERROR `SESSION_FULL`.

0x02 JOIN

Hôte → client

L'hôte admet le client et lui donne son ID.

En-tête	ID	HOST_NAME_LEN	HOST_NAME	REQUEST_ID
8 o	1 o	1 o	≤ 32 o	8 o (écho)

- Un NEW répété avec le même REQUEST_ID reçoit le **même** JOIN, sans créer de second participant.
- À réception, le client se considère connecté.

0x03 SYNCTABLE

Hôte → tous les clients

Liste complète des participants, hôte compris.

En-tête	HOSTS	ID	NAMELENGTH	NAME	… × HOSTS
8 o	1 o	1 o	1 o	≤ 32 o	

- Envoyée à chaque changement **et toutes les 5 s** : elle sert aussi de signe de vie de l'hôte.
- Les IP et ports ne sont jamais diffusés.

0x04 AVAILABLE

Client → hôte

« Je suis toujours là. » Envoyé toutes les 5 s.

Payload vide · LENGTH = 0

- L'hôte identifie le client par son endpoint et met à jour son dernier signe de vie.
- Sans nouvelles depuis 15 s, l'hôte retire le client et diffuse SYNCTABLE.

0x05 BYE

Client → hôte

Départ volontaire.

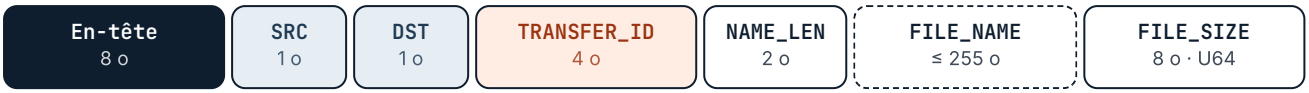
Payload vide · LENGTH = 0

- L'hôte retire le client, interrompt ses transferts et diffuse SYNCTABLE.

0x06 READY

Émetteur → hôte → destinataire

Annonce d'un fichier et de son destinataire.



- L'hôte vérifie : SRC = endpoint observé, DST connecté, 1er octet du TRANSFER_ID = SRC, TRANSFER_ID inédit. Puis il mémorise le routage.
- FILE_NAME est un nom seul, sans chemin.

0x07 YES_READY

Destinataire → hôte → émetteur

Le destinataire accepte le fichier.

En-tête	TRANSFER_ID
8 o	4 o

- Refus : ERROR `TRANSFER_REJECTED` à la place.
- Dans le MVP, l'application peut accepter automatiquement.

0x08 TAKE

Émetteur → hôte → destinataire

Transporte un bloc du fichier.

En-tête	TRANSFER_ID	SEQ	TOTAL	DATA
8 o	4 o	4 o	4 o	0 à 1 024 o

- Taille des données : DATA = LENGTH − 12.
- Pas de SRC/DST : l'hôte route avec le TRANSFER_ID enregistré lors de READY.

0x09 NICE

Destinataire → hôte → émetteur

Accusé de réception d'un bloc précis.

En-tête	TRANSFER_ID	SEQ
8 o	4 o	4 o

- Seul le NICE du destinataire vaut confirmation ; l'hôte n'en fabrique jamais.
- Un NICE dont le TRANSFER_ID ou le SEQ ne correspond pas est ignoré.

0x0A FINISH

Émetteur → hôte → destinataire

Tous les blocs sont confirmés ; voici l'empreinte.

En-tête	TRANSFER_ID	SHA256
8 o	4 o	32 o

- SHA-256 des octets du fichier dans l'ordre. Vérifie l'intégrité, ne chiffre rien.

0x0B COMPLETE

Destinataire → hôte → émetteur

Fichier reconstruit et empreinte vérifiée.

En-tête	TRANSFER_ID
8 o	4 o

- Empreinte différente → ERROR `HASH_MISMATCH`.

0x0C ERROR

Selon le contexte

Signale un problème.

En-tête	ERROR_CODE	TRANSFER_ID
8 o	2 o · U16	4 o

- Structure fixe de 6 octets. TRANSFER_ID = 0 si l'erreur ne concerne aucun transfert.

0x0D CANCEL

Émetteur → hôte → destinataire

Annule un transfert. Seul l'émetteur peut annuler.

En-tête	TRANSFER_ID
8 o	4 o

› Pas d'accusé : envoyé 3 fois. Le recevoir plusieurs fois est sans effet.

0x0E EXIT

Hôte → tous les clients

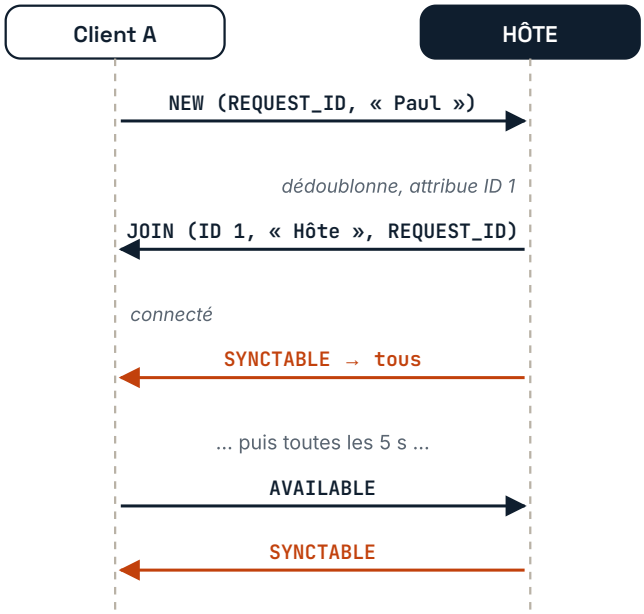
L'hôte ferme la session.

Payload vide · LENGTH = 0

› Envoyé 3 fois avant la fermeture du socket. Les clients arrêtent tout et suppriment leurs fichiers partiels.

8. Session et participants

Connexion : 3 étapes



La connexion tient en **NEW → JOIN → SYNCTABLE**. Si NEW ou JOIN se perd, le client renvoie NEW avec le même REQUEST_ID ; l'hôte reconnaît la demande et renvoie le même JOIN.

État	Signification
CONNECTED	JOIN envoyé · client admis
DISCONNECTED	BYE reçu ou 15 s sans AVAILABLE

Signes de vie croisés

5 s	Le client envoie AVAILABLE
5 s	L'hôte diffuse SYNCTABLE
15 s	Silence → l'autre est considéré parti

Côté client : 15 s sans aucun datagramme de l'hôte = hôte perdu. Le client interrompt ses transferts et revient à l'accueil. Ainsi la disparition brutale de l'hôte est détectée sans message supplémentaire.

Départs

BYE · UN CLIENT PART

L'hôte le retire, interrompt ses transferts (l'autre partie reçoit ERROR `TRANSFER_FAILED`) et diffuse SYNCTABLE.

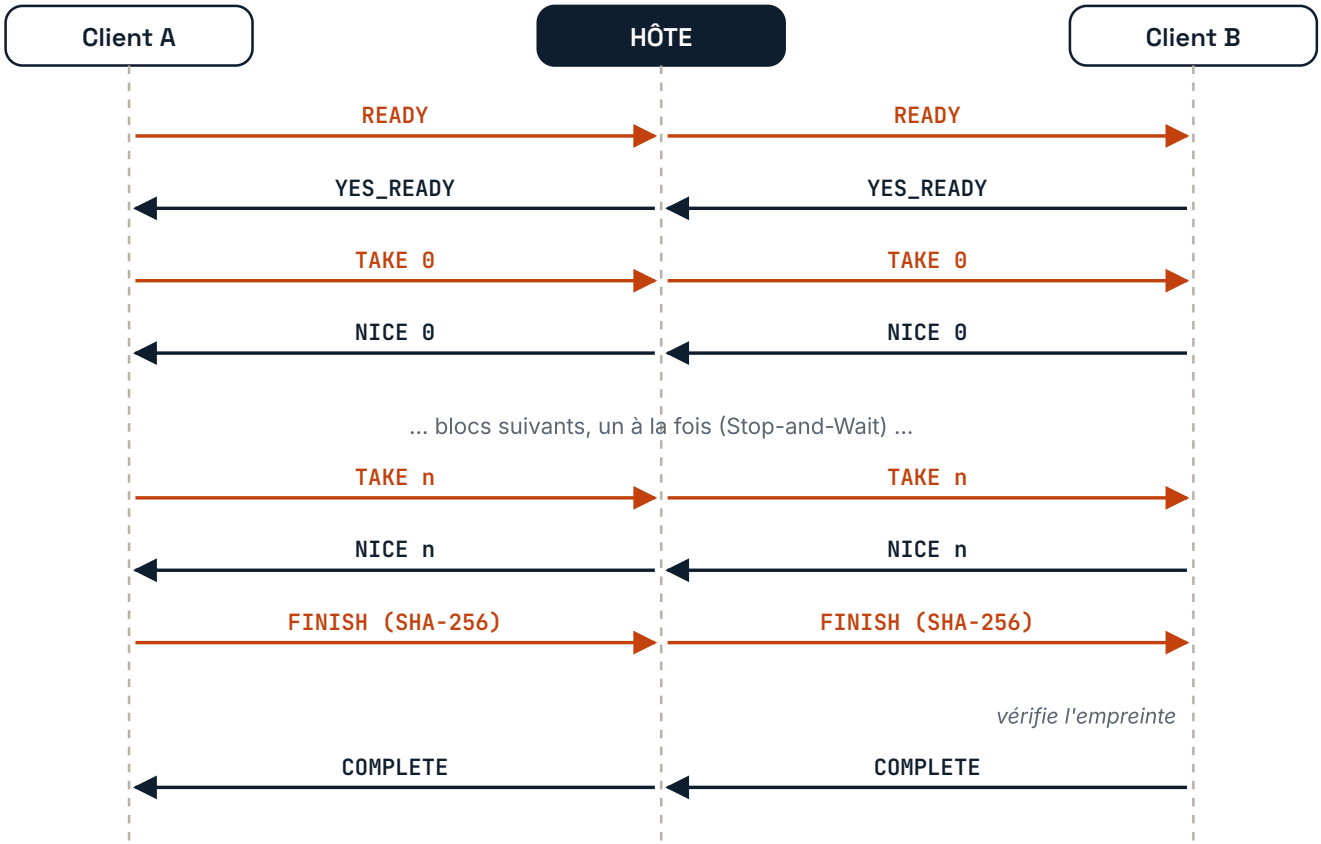
EXIT · L'HÔTE FERME

Envoyé 3 fois à tous. Chaque client arrête ses transferts, supprime ses fichiers partiels et ferme son socket.

Capacité : 8 participants au plus (hôte + 7 clients). Les IDs ne sont pas réutilisés dans une session ; une nouvelle session recommence à 1.

9. Transfert de fichiers

Scénario complet



Découpage en blocs

rapport.pdf · 3 000 octets

SEQ 0	SEQ 1	SEQ 2
1 024 o	1 024 o	952 o

`TOTAL = (FILE_SIZE + 1023) / 1024` · division entière · SEQ va de 0 à TOTAL - 1

Taille	TOTAL	Blocs
0 o	0	aucun TAKE : FINISH directement
1 o	1	1
1 024 o	1	1 024
1 025 o	2	1 024 + 1
3 000 o	3	1 024 + 1 024 + 952

Stop-and-Wait : un bloc à la fois

L'émetteur envoie un bloc, puis **attend son NICE** avant le suivant. C'est plus lent qu'une fenêtre glissante, mais très simple : un seul bloc en mémoire, la progression est évidente et les blocs arrivent forcément dans l'ordre. Le destinataire peut donc écrire le fichier au fil de l'eau et calculer le SHA-256 en même temps.

CÔTÉ DESTINATAIRE, À CHAQUE TAKE

- **SEQ = attendu** → écrire, envoyer NICE, attendre SEQ + 1
- **SEQ < attendu** → doublon : ne pas réécrire, renvoyer NICE
- **SEQ > attendu** ou TOTAL différent → ignorer
- DATA > 1 024 o ou transfert inconnu → ignorer

DOUBLONS : COMPORTEMENT ATTENDU

Reçu en double	Réaction
NEW même REQUEST_ID	renvoyer le même JOIN
READY même TRANSFER_ID	renvoyer la réponse connue
TAKE même SEQ	renvoyer NICE, sans réécrire
NICE déjà traité	ignorer
FINISH après la fin	renvoyer COMPLETE
CANCEL déjà appliqué	rester annulé

États d'un transfert

`REQUESTED` → `WAITING_ACCEPT` → `SENDING` ⇄ `WAITING_ACK` → `VERIFYING` → `COMPLETED`

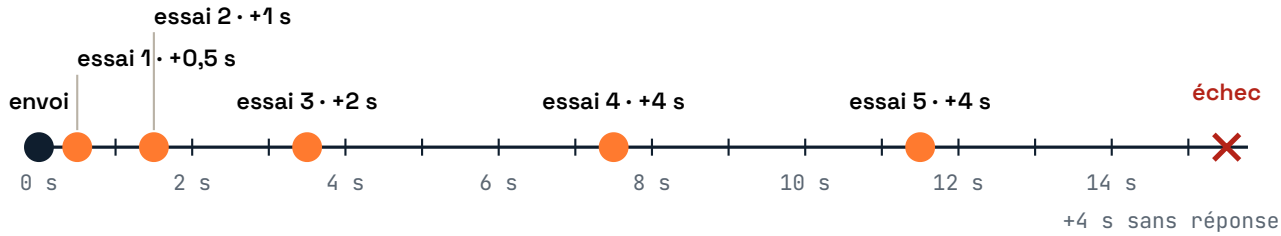
À tout moment : `CANCELLED` CANCEL valide `FAILED` ERROR, timeout ou départ d'un participant

État	Déclencheur	Action
REQUESTED	READY reçu par l'hôte	vérifier SRC, DST, TRANSFER_ID ; enregistrer le routage
WAITING_ACCEPT	READY relayé	attendre YES_READY ou ERROR
SENDING	YES_READY reçu	envoyer le bloc suivant
WAITING_ACK	TAKE envoyé	garder le bloc, attendre son NICE
VERIFYING	FINISH reçu	vérifier taille et SHA-256
COMPLETED	empreinte correcte	envoyer COMPLETE, libérer les ressources

Le destinataire abandonne aussi un transfert s'il ne reçoit plus rien pendant **15 s** (ERROR `TRANSFER_TIMEOUT`).

10. Retransmissions et délais

Si la réponse attendue n'arrive pas à temps, l'émetteur **renvoie le même message**, avec les mêmes identifiants, en doublant le délai à chaque essai.

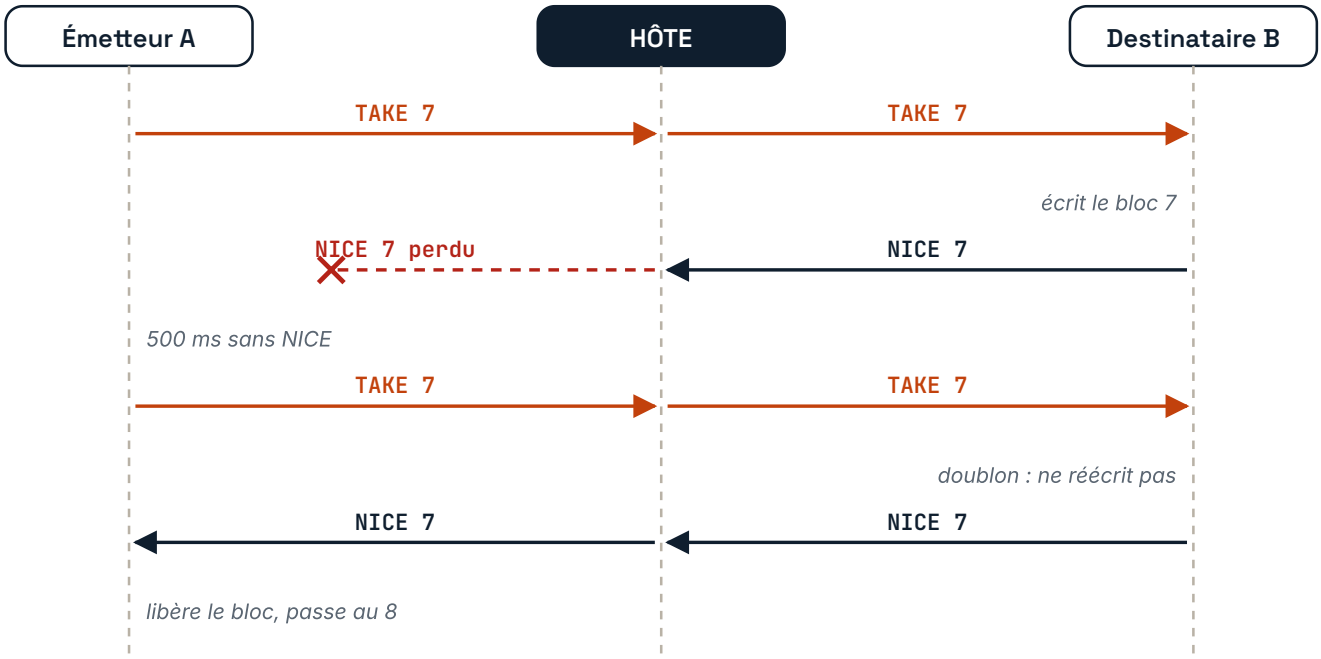


Envoi initial + 5 essais. Sans réponse après ~15,5 s, l'opération échoue : le transfert passe à FAILED (`TRANSFER_TIMEOUT`).

Paramètre	Valeur v1
Délai initial (RTO)	**500 ms**
Progression	× 2 à chaque essai
Plafond	4 s
Essais max.	5 (après l'envoi initial)
AVAILABLE / SYNCTABLE	toutes les 5 s
Expiration	15 s de silence

Message envoyé	Réponse attendue
NEW	JOIN
READY	YES_READY ou ERROR
TAKE	NICE du même SEQ
FINISH	COMPLETE ou ERROR
CANCEL, EXIT	aucune · envoyé 3 fois
AVAILABLE, BYE	aucune

Perte d'un NICE



Rôle de l'hôte — il relaie TAKE et NICE sans les confirmer lui-même. Un TAKE reçu par l'hôte n'est pas un TAKE reçu par le destinataire : seul le NICE du destinataire libère le bloc.

11. Erreurs et interruptions

Code	Nom	Envoyé par	Quand
`0x0001`	INVALID_MESSAGE	Tout participant	Structure incohérente (si une réponse est sûre)
`0x0002`	UNKNOWN_PARTICIPANT	Hôte	DST absent ou endpoint inconnu
`0x0003`	UNKNOWN_TRANSFER	Hôte ou destinataire	TRANSFER_ID inconnu ou expiré
`0x0004`	TRANSFER_REJECTED	Destinataire	Refus du fichier (au lieu de YES_READY)
`0x0005`	HASH_MISMATCH	Destinataire	SHA-256 différent de celui annoncé
`0x0006`	TRANSFER_TIMEOUT	Celui qui abandonne	Essais épuisés ou 15 s de silence
`0x0007`	TRANSFER_FAILED	Hôte ou participant	Échec générique, départ d'une des parties
`0x0008`	INVALID_SEQUENCE	Destinataire	Numéro de bloc incohérent (optionnel)
`0x0009`	SESSION_FULL	Hôte	NEW reçu alors que 8 participants sont présents

Le registre est extensible ; un code attribué n'est jamais réutilisé. Un code inconnu est traité comme une erreur générique, sans crash.

CANCEL ARRIVE DANS LE DÉSORDRE

UDP peut livrer un TAKE après le CANCEL. Le destinataire garde l'état « annulé » pour ce TRANSFER_ID et ignore les blocs tardifs. Un CANCEL répété ne réactive rien.

UN PARTICIPANT DISPARAÎT

Les transferts qui l'impliquent sont interrompus. L'hôte libère le routage et prévient l'autre partie (TRANSFER_FAILED). Le fichier partiel est supprimé.

L'HÔTE FERME OU DISPARAÎT

EXIT (×3) ou 15 s sans SYNCTABLE : les clients arrêtent tout, suppriment les fichiers partiels et reviennent à l'accueil.

PAQUET MALFORMÉ

Ignoré. On peut répondre INVALID_MESSAGE si c'est sûr, mais **jamais** à un ERROR : sinon deux appareils peuvent s'envoyer des erreurs en boucle.

12. Robustesse et sécurité

CONTRÔLES OBLIGATOIRES

- Taille minimale avant toute lecture
- MAGIC, VERSION, TYPE, FLAGS
- Taille réelle = 8 + LENGTH
- Longueurs de chaînes avant décodage UTF-8
- SRC déclaré = endpoint observé
- DST existant et connecté
- Seul SRC peut envoyer CANCEL
- Jamais d'allocation fondée sur une longueur non vérifiée

Limites assumées de la v1 — pas de chiffrement ni d'authentification. Un appareil connecté au même hotspot pourrait lire ou injecter des datagrammes. SHA-256 détecte une altération accidentelle, mais ne prouve pas qui a envoyé le fichier. Une version sécurisée devra le spécifier explicitement (échange de clés, chiffrement authentifié) — jamais via les FLAGS.

13. Décisions v1 et versionnage

Les points ouverts du brouillon sont tranchés. Ces valeurs font foi pour l'implémentation ; les tests diront lesquelles ajuster.

Sujet	Décision v1
MAGIC	« HRP » · `48 52 50` · en-tête de 8 octets
VERSION · FLAGS	`01` · `00` obligatoire, sinon paquet ignoré
Port	50000/UDP sur l'hôte
JOIN	contient l'écho du REQUEST_ID
Connexion	NEW → JOIN → SYNCTABLE (AVAILABLE n'en fait plus partie)
Signes de vie	AVAILABLE et SYNCTABLE toutes les 5 s · expiration 15 s
Perte de l'hôte	détectée par 15 s sans SYNCTABLE ni autre datagramme
Retransmissions	500 ms, × 2, plafond 4 s, 5 essais
TRANSFER_ID	(ID émetteur × 2^{24}) + compteur, compteur dès 1 ; jamais 0
ERROR	6 octets fixes ; TRANSFER_ID = 0 hors transfert
CANCEL · EXIT	sans accusé, envoyés 3 fois ; fichiers partiels supprimés
Capacité	8 participants ; IDs non réutilisés dans une session
Limites	nom de participant ≤ 32 o · nom de fichier ≤ 255 o
Fichier vide	TOTAL = 0, FINISH directement
Nouvelle erreur	`0x0009` SESSION_FULL

Versionnage

v1.0
Spécification figée
12 nov. 2026

→

v1.1
Ajustements après tests
(délais, limites)

→

v2
Évolutions : fenêtre glissante, découverte, sécurité

Une correction qui ne change pas le format des paquets → v1.x. Un changement de format → VERSION passe à `02`, et un appareil v1 ignore les paquets v2.

Hors périmètre de la v1

Internet, cloud ou relais externe · iOS, web, ordinateur · chiffrement et authentification · reprise d'un transfert après fermeture de l'application · fenêtre glissante · découverte automatique de l'hôte.

Annexe A · Structures condensées

Code	Message	Payload	Min.
01	NEW	REQUEST_ID[8] · NAMELENGTH[1] · NAME[N]	9 o
02	JOIN	ID[1] · HOST_NAME_LEN[1] · HOST_NAME[N] · REQUEST_ID[8]	10 o
03	SYNCTABLE	HOSTS[1] · (ID[1] · NAMELENGTH[1] · NAME[N]) × HOSTS	1 o
04	AVAILABLE	vide	0
05	BYE	vide	0
06	READY	SRC[1] · DST[1] · TRANSFER_ID[4] · NAME_LEN[2] · FILE_NAME[N] · FILE_SIZE[8]	16 o
07	YES_READY	TRANSFER_ID[4]	4 o
08	TAKE	TRANSFER_ID[4] · SEQ[4] · TOTAL[4] · DATA[0..1024]	12 o
09	NICE	TRANSFER_ID[4] · SEQ[4]	8 o
0A	FINISH	TRANSFER_ID[4] · SHA256[32]	36 o
0B	COMPLETE	TRANSFER_ID[4]	4 o
0C	ERROR	ERROR_CODE[2] · TRANSFER_ID[4]	6 o
0D	CANCEL	TRANSFER_ID[4]	4 o
0E	EXIT	vide	0

N = longueur indiquée par le champ de longueur qui précède.

Annexe B · Guide d'implémentation Java

- Une classe dédiée pour construire et lire l'en-tête
- `ByteBuffer` en `BIG_ENDIAN` explicite
- Constantes ou `enum` pour TYPE et ERROR_CODE
- Vérifier la longueur avant chaque `get()`
- Séparer le parseur réseau de la logique Android
- Générer REQUEST_ID une fois par tentative (`SecureRandom`)
- SHA-256 incrémental avec `MessageDigest`
- Jamais d'UDP sur le thread UI : un thread dédié
- Un état explicite par transfert
- Journaliser TYPE, TRANSFER_ID, SEQ, heure — pas le contenu

```
static final byte[] MAGIC = { 0x48, 0x52, 0x50 }; // "HRP"
static final int HEADER_SIZE = 8, PORT = 50000, MAX_DATA = 1024;

int transferId(int senderId, int counter) {
    return (senderId << 24) | (counter & 0xFFFFFF);
}
```

Annexe C · Checklist de tests

- [ ] NEW valide, NEW dupliqué (même REQUEST_ID), NEW d'une autre tentative
- [ ] SYNCTABLE avec plusieurs noms accentués
- [ ] Disparition brutale de l'hôte détectée en 15 s
- [ ] TAKE de 1 o, 1 024 o, moins de 1 024 o
- [ ] FINISH avec empreinte correcte et incorrecte
- [ ] CANCEL par l'émetteur, par un autre, TAKE tardif
- [ ] Datagramme tronqué, LENGTH faux, TYPE / VERSION inconnus, FLAGS ≠ 0
- [ ] Session pleine → SESSION_FULL
- [ ] AVAILABLE, expiration à 15 s, BYE
- [ ] READY accepté, refusé, dupliqué, destinataire inconnu
- [ ] Perte de TAKE, perte de NICE, doublons
- [ ] Fichier vide, fichier multiple exact de 1 024
- [ ] EXIT pendant un transfert
- [ ] Deux transferts simultanés vers des destinataires différents